# ALGORITMI DI ORDINAMENTO

Uno dei problemi più comuni e classici dell'informatica è la produzione di una **permutazione** degli elementi che rispetti la relazione. Ovvero, disporre gli elementi di un insieme *A* secondo una prefissata **relazione d'ordine**.
A seconda dell'informazione analizzata si hanno diversi tipi di relazione d'ordine, tra cui:

- Numerica
- Alfanumerica
- Altri criteri

Esistono numerosi algoritmi, basati su confronti e scambi tra elementi, per la gestione delle relazioni d'ordine. Nessuno di questi è perfetto, l'efficacia di un algoritmo rispetto ad un altro dipende dai **dati** su cui esso dev'essere applicato e dal **grado di ordinamento**, ovvero se è già ordinato, in parte o gli elementi dell'insieme sono posti casualmente.

Creare collezioni di dati che dispongono di una struttura ordinata è efficace nei calcolatori perché porta a differenti benefici, tra cui l'accesso alle collezioni stesse e ai loro elementi, permette di usare algoritmi di ricerca più efficienti e perfeziona la **valenza applicativa** (circa il 30% del tempo di calcolo di un elaboratore).
Tutto ciò mira a rendere più efficiente il prodotto. Poiché si sfruttano al meglio i confronti e gli spostamenti.

In ogni algoritmo di ordinamento bisogna tenere contò di un particolare dettaglio, **lo scambio è più costoso del confronto**; dato che il **confronto** è un operazione base del processore, differentemente lo **scambio** è l'unione di diversi assegnamenti.

Gli algoritmi di ordinamento sono:

- **ESTERNI**
- **INTERNI**

# ORDINAMENTO ESTERNO

Nell'ordinamento esterno gli algoritmi si basano sull'uso di array di **appoggio**, dove trasportare momentaneamente i dati per poi essere riscritti sull'array originale, portando anche ad un occupazione doppia della memoria.
Uno dei più famosi algoritmi esterni è il **Counting Sort** (Sort Enumerativo).

## COUNTING SORT

Nel counting sort vogliamo determinare le posizioni dell'ordinamento. Ogni elemento di $a_j$ dell'insieme *A* viene confrontato con gli altri elementi di *A*. Si determineranno quanti elementi

sono più piccoli di $a_j$. Se *k* elementi sono più piccoli di $a_j$, la posizione finale di tale valore nell'insieme ordinato sarà *k+1*.
Ad esempio se ci sono 3 elementi < $a_j$, la sua posizione sarà 4.
Per trovare i *k* elementi più piccoli, si utilizza un array in cui registrare gli elementi presenti nell'insieme.
Dopo aver determinato il valore di *k* per ogni elemento, si costruisce un array contenente ogni elemento nella sua posizione corretta.

L'algoritmo si basa sul **conteggio delle occorrenze** e non sul confronto diretto di tutti gli elementi.

*Esercizio di comprensione sul **Counting sort***:
**Array iniziale A=[4,2,1,3]**
Confrontiamo gli elementi:

- 4 ha 3 numeri più piccoli
- 2 ha 1 numero più piccolo
- 1 ha 0 numeri più piccoli
- 3 ha 2 numeri più piccoli
  Seguendo la legge **k+1**, possiamo riordinarli dicendo che:
- 3+1=4 -> posizione di 4
- 1+1=2 -> posizione di 2
- 0+1=1 -> posizione di 1
- 2+1=3 -> posizione di 3
  Il nuovo array sarà **A=[1,2,3,4]**

# ORDINAMENTO INTERNO

Eseguono l'ordinamento sul medesimo array che possiedono. La logica è basata su:

- **Selezione**: localizzare l'elemento più piccolo e separarlo e così via.
- **Inserzione**: esaminare un elemento per volta e posizionarlo al posto che gli compete.
- **Scambio**: Scambiare di posizione coppie di elementi finché non risultino tutti ai loro posti e non sarà più necessario attuare altri scambi.

## ORDINAMENTO PER SELEZIONE (SELECTION-SORT):

Utilizziamo il **Selection-sort** per ordinare gli n elementi di un array.
L'algoritmo per minimi successivi seleziona l'elemento con valore minore e lo scambia col primo elemento.
Si eseguirà il medesimo passaggio con l'elemento *n-1*, fino al raggiungimento dell'ultimo elemento.

*Esempio del lavoro del **Selection-sort***:

| original array | after pass 1 | after pass 2 | after pass 3 | after pass 4 | after pass 5 |
|---|---|---|---|---|---|
| 3 | 1 | 1 | 1 | 1 | 1 |
| 6 | 6 | 3 | 3 | 3 | 3 |
| 1 | 3 | 6 | 4 | 4 | 4 |
| 8 | 8 | 8 | 8 | 5 | 5 |
| 4 | 4 | 4 | 6 | 6 | 6 |
| 5 | 5 | 5 | 5 | 8 | 8 |

Nel codice questa procedura viene tradotta in questo modo:

```
{
        for(i=0;i<n-1;i++){
                min=a[i]
                p=i;
                for(j=i+1;j<n;j++){
                        if(a[j]<min){
                                min=a[j];
                                p=j;
                        }
                }
                a[p]=a[i];
                a[i]=min;
```

```
        }
}
```

Nella prima iterazione bisogna effettuare n-1 confronti per determinare l'elemento più piccolo.
Nella seconda iterazione bisogna effettuare n-2 confronti per determinare il minimo tra gli n-1 elementi e così sarà per tutte le altre iterazioni.
Si effettuerà il controllo tra 2 elementi per ricavarne il minimo.

Possiamo, in questo modo, dichiarare che il numero dei confronti che si andranno ad attuare è pari a:

$\sum_{i=1}^{n-1} n-1 = (n-1)+(n-2)+\ldots+1 = n(n-1)/2$

Il tempo di esecuzione dell'algoritmo tende a $n^2$.
Il comportamento del selection-sort è indipendente da come sono disposti gli elementi inizialmente nell'array.

Possiamo considerare che l'algoritmo non sfrutta un eventuale preordinamento ma eseguirà in tutti i casi tutti gli *n* confronti, anche se l'array dov'esse essere già ordinato. Sarebbe utile evitare gli scambi con se stessi, come nel caso in cui il valore di *p* sia lo stesso di *i*.

## ORDINAMENTO PER INSERIMENTO (INSERTION-SORT)

L'algoritmo **insertion-sort** si basa sul metodo usato per reinserire le carte di un mazzo mentre si sta mischiando quest'ultimo.

Per ricreare questo algoritmo su un array di interi, in ordine crescente, si considera il primo elemento come un **sottovettore ordinato** (poiché essendo solo un elemento dell'insieme sarà per forza ordinato).
Attraverso l'uso delle regole della matematica discreta, opereremo come relazione tra sotto insiemi, **singoletti**. Prenderemo l'elemento successivo ,*a[2]*,
e verrà inserito nella giusta posizione rispetto al sottovettore ordinato. Ripeteremo questa operazione per ogni elemento ***k***, inserendoli al posto giusto tra i primi *k-1* elementi già ordinati.

**Ma come si inserisce l'elemento al posto giusto?**
Per inserire l'elemento al posto giusto si suppone di avere ordinato i primi *k-1* elementi, per aggiungere il nuovo elemento *k* bisogna **confrontarlo con i suoi predecessori volta per volta**, spostando gli elementi più grandi verso destra, fino a trovare la posizione corretta per l'elemento *k*.
L'elemento da **inserire al posto giusto** viene definito **chiave**.

*Esempio del lavoro del **Insertion-sort***:
Supponiamo di avere *a[5,3,8,6]* da disporre in ordine crescente.
L'elemento *a[1]* sarà il sottovettore già ordinato (contenente il valore 5).

Successivamente prenderemo il secondo elemento (3), poiché 3<5 spostiamo 5 a destra e 3 sarà inserito nella sua posizione corretta.
Ora avremmo un array differente: *a[3,5,8,6]*, ripetiamo le operazioni precedenti per tutti gli altri elementi dell'array; arrivando in fine ad avere un array ordinato di questo tipo: ***a[3,5,6,8]***.

Questa procedura nel codice viene tradotta con:

```
{
        for(i=1;i<n;i++){
                x=a[i];
                j=i-1;
                        while((j>=0)&&(a[j]>x)){
                                a[j+1]=a[j];
                                j=j-1;
                        }
                a[j+1]=x;
        }
}
```

Possiamo evincere che l'ordinamento per inserzione, inserisce gli elementi lavorando meno se essi sono già parzialmente ordinati. Se il vettore di partenza è già ordinato si entrerà nel caso **migliore**, in cui basteranno n-1 confronti.
Il caso **peggiore** avverrà solamente se la sequenza degli elementi è ordinata in maniera **decrescente**, in cui si dovranno eseguire tutti i confronti.

## ORDINAMENTO A SCAMBIO (BUBBLE-SORT)

L'ordinamento a bolle è un algoritmo basato sul metodo di **scambi**.
Gli elementi più piccoli **salgono** verso l'inizio del vettore scambiandosi con quelli adiacenti.
Si lavora a **coppie**, scambiando il posto degli elementi della coppia ogni qualvolta che essa non sia ordinata.

Per esempio dato il vettore A[n], confrontiamo se l'elemento A[1] è maggiore di A[2], in caso positivo si effettua uno scambio tra i due. Ciò sarà ripetuto fino all'elemento A[n-1] confrontato con A[n], scambiando il posto con gli elementi che non formano coppie ordinate.
Si ripeterà questo ciclo fino a quando non ci saranno più valori da scambiare.

Il bubble-sort, sotto alcuni punti di vista, è **efficiente** poiché permette di eseguire l'algoritmo in **meno di *n-1* passi**, infatti se non è stato effettuato alcuno scambio, l'insieme è già **ordinato**.
Sfrutta a pieno gli ordinamenti già presenti nell'array di partenza.
Inoltre questo algoritmo tiene traccia degli scambi effettuati attraverso un **indicatore**. Il quale è impostato a true e muterà in false solamente quando avverrà uno scambio. Se l'indicatore rimane **inalterato** dopo un intera passata allora l'algoritmo termina.

***Rappresentazione del bubble-sort:***

Esempio – Prima passata

| | Inizio | I/1 | I/2 | I/3 | I/4 | II |
|---|---|---|---|---|---|---|
| *a(1)* | 4 | 4 | 4 | 4 | 4 | 1 |
| *a(2)* | 3 | 3 | 3 | 3 | 1 | 4 |
| *a(3)* | 6 | 6 | 6 | 1 | 3 | 3 |
| *a(4)* | 1 | 1 | 1 | 6 | 6 | 6 |
| *a(5)* | 5 | 2 | 2 | 2 | 2 | 2 |
| *a(6)* | 2 | 5 | 5 | 5 | 5 | 5 |

1.

Esempio – Seconda passata

| | II | II/1 | II/2 | II/3 | III |
|---|---|---|---|---|---|
| *a(1)* | 1 | 1 | 1 | 1 | 1 |
| *a(2)* | 4 | 4 | 4 | 4 | 2 |
| *a(3)* | 3 | 3 | 3 | 2 | 4 |
| *a(4)* | 6 | 6 | 2 | 3 | 3 |
| *a(5)* | 2 | 2 | 6 | 6 | 6 |
| *a(6)* | 5 | 5 | 5 | 5 | 5 |

2.

Esempio – Terza e Quarta passata

| | III | III/1 | III/2 | IV | IV/1 | Fine |
|---|---|---|---|---|---|---|
| *a(1)* | 1 | 1 | 1 | 1 | 1 | 1 |
| *a(2)* | 2 | 2 | 2 | 2 | 2 | 2 |
| *a(3)* | 4 | 4 | 4 | 3 | 3 | 3 |
| *a(4)* | 3 | 3 | 3 | 4 | 4 | 4 |
| *a(5)* | 6 | 5 | 5 | 5 | 5 | 5 |
| *a(6)* | 5 | 6 | 6 | 6 | 6 | 6 |

3.

La sottosequenza ancora da ordinare si riduce di un elemento ad ogni scansione.

Questa procedura nel codice viene tradotta con:

```
{ sorted = 0; p = 0;
  while (( p < n-1) && (sorted!=1))
    {
    sorted = 1;
    for (j = n;  j > p;  --j) {
        if (a[j] < a[j–1])  {
                t=a[j];
                a[j]=a[j–1];
                a[j–1]= t;
                sorted = 0;
        }
    }
     p = p + 1;
    }
}
```

```
{
    for (i = 0; i < n-1 && sorted!=1; i++) {
        sorted = 1;
        for (j = n-1; j > i; –j) {
            if (a[j] < a[j–1]) {
                t = a[j];
                a[j] = a[j–1];
                a[j–1] = t;
                sorted = 0
            }
        }
    }
}
```

Il Bubble-sort scorre solamente la parte **non ordinata**. Nel caso **peggiore** avremmo lo stesso problema dei confronti presenti nell'ordinamento per selezione ma con un numero di scambi molto più elevato, tuttavia è innegabile quanto sia **veloce** e performante per gli insiemi con alto grado di preordinamento. La scansione degli elementi è definita **passata**.

## ESERCIZI BUBBLE SORT:

1. Data il seguente array //$a[12, 18, 42, 44, 55, 67, 94, 6]$// mostrare gli elementi dell'array dopo ogni iterazione del ciclo esterno e calcolare quante passate necessitano per ordinarlo.

   Come primo step implemento il codice con il seguente array:

```c
BubbleSort
*****************************************************************************/
#include <stdio.h>
#define N 8
int main()
{
    //definizione array e variabili di lavoro
    int a[N]={12,18,42,44,55,67,94,6}, sorted=0,t=0, p=0;

    for(int i=0;i<N-1 && sorted!=1;i++){ //Ciclo esterno
        sorted=1;
        //scambio degli elementi adiacenti
        for(int j=N-1;j>i;--j){
            if(a[j]<a[j-1]){
                t=a[j];
                a[j]=a[j-1];
                a[j-1]=t;
                sorted=0;
            }
        }
        p=i+1;

        //output array ad ogni iterazione del ciclo esterno
        printf("\nSiamo nel ciclo esterno all'iterazione %d. Con i seguenti valori", p);
        for(int o=0;o<N;o++){
            printf("\n %d", a[o]);
        }
    }

    //Passate effettuate
    printf("\nLe passate necessario per ordinarlo sono state: %d", p);
}
```

   L'output finale che riceveremo sarà il medesimo:

```
Siamo nel ciclo esterno all'iterazione 1. Con i seguenti valori
 6
 12
 18
 42
 44
 55
 67
 94
Siamo nel ciclo esterno all'iterazione 2. Con i seguenti valori
 6
 12
 18
 42
 44
 55
 67
 94
Le passate necessario per ordinarlo sono state: 2
```

Gli scambi da attuare saranno 7 per far passare il 6 dall'ultima posizione alla prima, ciò è l'operazione che eseguirà il primo ciclo esterno, dopo aver posizionato il 6 andrà a controllare gli altri valori, essendo i valori successivi in ordine la sentinella **sorted** sarà impostata ad 1 facendo terminare il ciclo esterno. Avendo di fatto questo nuovo array con solo **2 passate**: $a[6, 12, 18, 42, 44, 55, 67, 94]$

2. Data il seguente array //$a[94, 6, 12, 18, 42, 44, 55, 67]$// mostrare gli elementi dell'array dopo ogni iterazione del ciclo esterno e calcolare quante passate necessitano per ordinarlo.

   Eseguiamo nuovamente il codice precedente con i nuovi dati, ricevendo in output:

   Siamo nel ciclo esterno all'iterazione 1. Con i seguenti valori
   6
   94
   12
   18
   42
   44
   55
   67
   Siamo nel ciclo esterno all'iterazione 2. Con i seguenti valori
   6
   12
   94
   18
   42
   44
   55
   67
   Siamo nel ciclo esterno all'iterazione 3. Con i seguenti valori

6
12
18
94
42
44
55
67
Siamo nel ciclo esterno all'iterazione 4. Con i seguenti valori
6
12
18
42
94
44
55
67
Siamo nel ciclo esterno all'iterazione 5. Con i seguenti valori
6
12
18
42
44
94
55
67
Siamo nel ciclo esterno all'iterazione 6. Con i seguenti valori
6
12
18
42
44
55
94
67
Siamo nel ciclo esterno all'iterazione 7. Con i seguenti valori
6
12
18
42
44
55
67

94
Le passate necessario per ordinarlo sono state: 7

Come si nota le passate sono state maggiori, poiché ci sono stati molti più spostamenti per portare il 94 in ultima posizione, potevamo fermarci perché gli altri numeri erano già impostati alla loro corretta posizione, tuttavia il bubble-sort operando dal basso verso l'altro ha dovuto far salire tutti i numeri presenti al di sotto del numero massimo (94). Portandoci ad avere ben **7 passate**.

# MERGE

Un altro algoritmo di ordinamento è il **Merge**, anche detto Fusione.
La base di questo algoritmo è la possibilità di fondere due algoritmi già **ordinati** per creare un unico array ordinato, il quale seguirà sempre lo stesso criterio.
Prendiamo per esempio due array "$a[2, 5, 9, 13, 24]$" e "$b[3, 4, , 11, 15, 22]$", il nuovo array *c* sarà dato dalla somma degli elementi dei due array precedenti, sotto la formula: $m + n$.
L'algoritmo del merge analizza tutti gli elementi di tutti gli array analizzati e considera ogni array come **sottosequenze da concatenare**.

Si esamina il primo elemento della parte da fondere dei due array, nel nostro caso $a[2]$ e $b[3]$ e si consideri il minore tra i due. L'elemento più piccolo sarà posto come primo elemento dell'array *c* e si tornerà all'analisi degli elementi dall'array che possedeva l'elemento più piccolo.
Uno dei due array terminerà per **primo**, allora si copierà nel array definitivo gli altri elementi presenti nell'array non ancora terminato. Chi dei due terminerà prima è facilmente individuabile, basterà **confrontare** inizialmente l'ultimo valore del primo array con l'ultimo valore del secondo array e vedere qual è il valore **minore**.

Questo algoritmo non è ottimale per fondere due insiemi di dati di cui non si conosca la **cardinalità**, per questo si sono create delle modifiche al codice per adattarlo ai file sequenziali, creando un algoritmo di tipo **merge avanzato**.

# MERGESORT (FUSIONI SUCCESSIVE)

Si basa sul principio dei sotto problemi, ovvero quello del **divide et impera**.
La struttura di questo ordinamento si divide nella:

- **DIVISIONE** delle due liste dimezzandole perfettamente.
- **ORDINANDO** ogni singola sequenza, ed essendo scomposti la difficoltà di ordinamento diventerà sempre più semplice. Questa divisione in array sempre più piccoli sarà **ripetuta** fino all'ottenimento di un **sottoarray** già ordinato.
- **FONDENDO** in fine tutti i sottoarray ordinati ottenuti.
  Ricevendo così una lista ordinata anche avendo *n* elementi.

*Esempio MergeSort:*

- Insieme iniziale

| 33 | 21 | 7 | 48 | 28 | 13 | 65 | 17 |

- Suddivisione in 2 sottoinsiemi

| 33 | 21 | 7 | 48 |  | 28 | 13 | 65 | 17 |

- Ordinamento di ogni singolo sottoinsieme

| 7 | 21 | 33 | 48 |  | 13 | 17 | 28 | 65 |

- Fusione dei sottoinsiemi ordinati

| 7 | 13 | 17 | 21 | 28 | 33 | 48 | 65 |

Come possiamo vedere abbiamo anche eliminato l'obbligo di avere due array già ordinati.

Esiste un criterio per determinare la **ricorsività** del MergeSort:

1. Se l'array da ordinare contiene $n = 1$ o $n = 0$ elementi, l'array è già ordinato
2. Se l'array da ordinare contiene $n > 1$ elementi, l'array iniziale subirà i 3 passaggi dell'algoritmo del MergeSort

Questa procedura nel codice viene tradotta con:

```
void MergeSort(int a[ ], int primo, ultimo)
 {
  int mid;

 if (primo < ultimo)     /* l'array a ha almeno 2 elementi */
        {
         mid = (primo + ultimo) / 2;
         MergeSort(a, primo, mid);   /* prima call ricorsiva */
         MergeSort(a, mid + 1, ultimo);   /*seconda call ricorsiva*/
         merge (a, primo, mid, ultimo);   /* procedura iterativa */
         }
 }
```

```
void merge( int a[ ], int b[ ], int c[ ], int m, int n)
{
   int i = 0, j= 0, k = 0;

   while ((i < m)&& (j < n)) {
        if (a[i] < b[j])
                {c[k] = a[i];  i=i+1; k=k+1};
        else {c[k] = b[j]; j=j+1; k=k+1};
        }

   while (i < m)  c[k++] = a[ i++];
   while (j < n) c[k++] = b[ j++];
}
```

***void merge (int a[ ]; int primo, mid, ultimo)***
***/* fusione dei sottovettori a[primo..mid] e a[mid+1..ultimo] */***

Durante il processo di **divisione** gli elementi non vengono né spostati, né confrontati, la divisione viene eseguita fino all'arrivo di una sequenza di **dimensione 1**, di fatto quando la sottosequenza arriverà ad avere una dimensione pari ad 1, l'*if(primo<ultimo)* presente nel MergeSort sarà falso poiché $primo = ultimo$. La potenza di questo algoritmo si basa sul **backtracking** (l'operazione di **richiamo della funzione** per tornare indietro ad eseguire gli

stessi passi fin quando non si troverà una soluzione ottimale).
La complessità computazionale di questo algoritmo è ***O(nlog2 n)*** , infatti il tempo d'esecuzione è **più veloce** rispetto ad un algoritmo lineare, grazie al **backtracking**.
*O* sta ad indicare la **complessità di compilazione**.

In questo algoritmo **non esistono casi favorevoli**, l'algoritmo lavora sempre nello stesso modo, qualsiasi sia la disposizione degli elementi dei due array.

# TEST SPERIMENTALI

Un altro modo per confrontare le **prestazioni** degli algoritmi di ordinamento è eseguire numerosi **test**, aumentando progressivamente la dimensione dell'input. Durante questi test, vengono misurati i tempi di esecuzione degli algoritmi su vettori disposti in ordine crescente, decrescente e casuale. I tempi vengono generalmente espressi in minuti e secondi. Tuttavia, questo metodo presenta uno **svantaggio**: i tempi di esecuzione **dipendono dalle caratteristiche della macchina** su cui gli algoritmi vengono eseguiti, come la velocità del processore e la capacità della memoria. Di conseguenza, il tempo di esecuzione può variare notevolmente se l'algoritmo viene eseguito su macchine differenti.